

w5L27 L_{π} Convergence (just screen shots)

L_{π} is a contraction

$$Q^* = \max_{\pi} \{ r_{\pi} + \gamma P^{\pi} Q^* \} \triangleq L Q^*$$

$$Q^*(s) = \max_a \left\{ E[r|s, a] + \gamma \sum_j P(j|s, a) \cdot Q^*(j) \right\}$$

$$L Q \triangleq \max_{\pi} \{ r_{\pi} + \gamma P^{\pi} Q \}$$

(claim: L is a contraction.)

Let $a_s^* \in \arg \max_a \left\{ E[r|s, a] + \gamma \sum_j P(j|s, a) \cdot Q(j) \right\}$

$$0 \leq L Q(s) - L U(s) \leq E[r|s, a_s^*] + \gamma \sum_j P(j|s, a_s^*) \cdot Q(j) - \left\{ E[r|s, a_s^*] + \gamma \sum_j P(j|s, a_s^*) \cdot U(j) \right\}$$

L_v is a function operates on s and then gives a function again

$$\begin{aligned}
 &= \gamma \sum_j p(j|s, a_s^*) [v(j) - u(j)] \\
 &\leq \gamma \|v - u\| \cdot \sum_j \cancel{p(j|s, a_s^*)} \\
 0 &\leq Lv(s) - Lu(s) \leq \gamma \|v - u\| \\
 |Lv(s) - Lu(s)| &\leq \gamma \|v - u\| \quad \forall s \\
 \Rightarrow L &\text{ is a contraction}
 \end{aligned}$$

Value Iteration

1. Select $v^0 \in V$, pick $\alpha \in (0, 1)$. Set $n = 0$

2. For each $s \in S$, compute $v^{n+1}(s)$ by

$$v^{n+1}(s) = \max_a \left\{ E[r|s, a] + \gamma \sum_{s'} p(s'|s, a) \cdot v^n(s') \right\}$$
$$\{v^{n+1}\} = L v^n$$

3. If $\|v^{n+1} - v^n\| < \epsilon \cdot (1 - \gamma) / 2\gamma$,

go to step 4, or increment n by 1

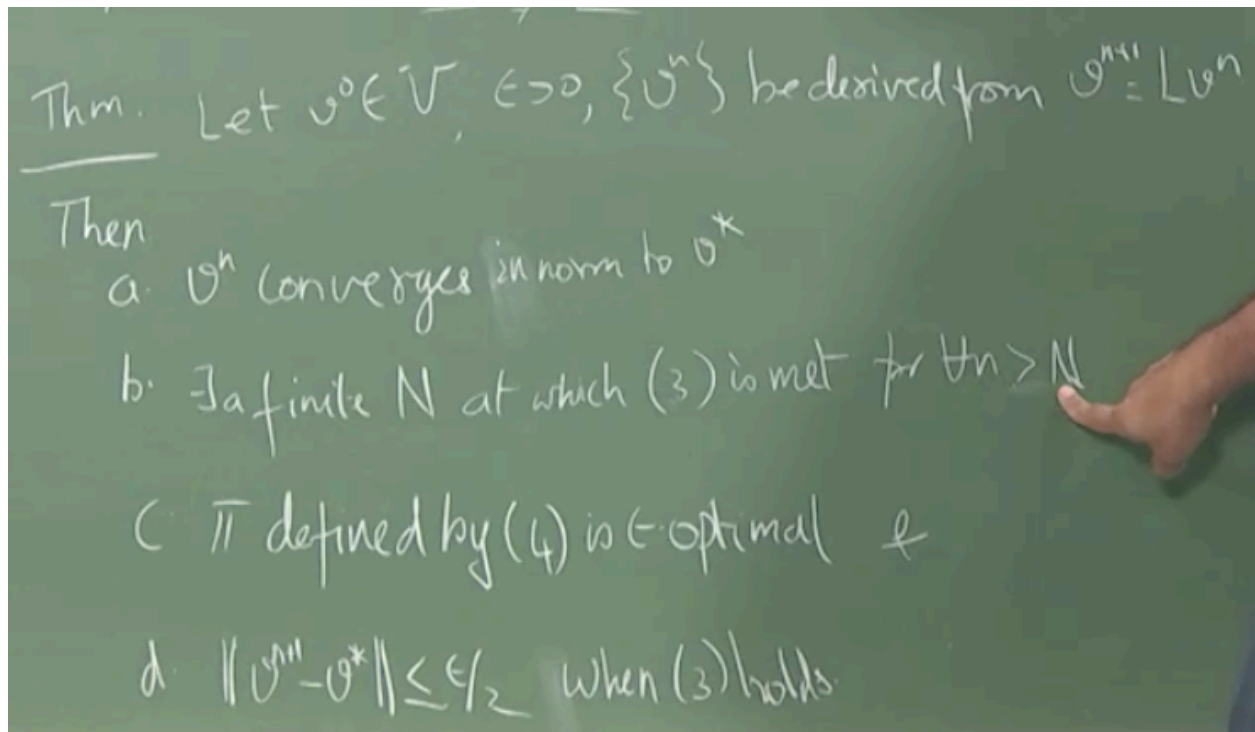
go to step 2

4. For each $s \in S$,

$$\pi(s) = \arg \max_a \left\{ E[r|s, a] + \gamma \sum_{s'} p(s'|s, a) \cdot v^{n+1}(s') \right\}$$

Stop

w5-L28 Value Iteration Proof (31min)



A and b are Banach Theorem

What is the difference between C and D

- Do they look redundant

It is epsilon optimal

$$v - \pi - v^* < \epsilon$$

C is saying v^* is epsilon optimal

D is saying v^{n+1} is epsilon/2 optimal

v is NOT space of value function

$L - \pi$ acting on v^{n+1}

or each $s \in P$,

$$\pi(s) = \arg \max_a \left\{ E[x | s, a] - \gamma \sum_{s'} p(s' | s, a) \cdot v^{n+1}(s') \right\}$$

(c)
 Suppose (3) is met for some n , & π satisfies (2). Then:

$$\|v^\pi - v^*\| \leq \|v^\pi - v^{n+1}\| + \|v^{n+1} - v^*\|$$

$$L_{\pi} v^{n+1} = E[r|s, \pi(s)] + \gamma \sum_{s'} p(s'|s, \pi(s)) v^{n+1}(s')$$

$$L v^{n+1} = L_{\pi} v^{n+1}$$

Simplify

$$\|v^\pi - v^*\| \leq \|v^\pi - v^{n+1}\| + \|v^{n+1} - v^*\|$$

$$L_{\pi} v^{n+1} = E[r|s, \pi(s)] + \gamma \sum_{s'} p(s'|s, \pi(s)) v^{n+1}(s')$$

$$L v^{n+1} = L_{\pi} v^{n+1}$$

$$\|v^\pi - v^{n+1}\| = \|L_{\pi} v^\pi - v^{n+1}\|$$

$$L_{\pi}^{n+1} = E[r|s, \pi(s)] + \gamma \sum_{s'} p(s'|s, \pi(s)) V^{n+1}(s')$$

$$L_{\pi}^{n+1} = L_{\pi}^{n+1}$$

$$\| \pi - \pi^{n+1} \| = \| L_{\pi}^{\pi} - \pi^{n+1} \|$$

$$\| \pi - \pi^{n+1} \| = \| L_{\pi}^{\pi} - \pi^{n+1} \|$$

$$\leq \| L_{\pi}^{\pi} - L_{\pi}^{\pi^{n+1}} \| + \| L_{\pi}^{\pi^{n+1}} - \pi^{n+1} \|$$

$$= \| L_{\pi}^{\pi} - L_{\pi}^{\pi^{n+1}} \| + \| L_{\pi}^{\pi^{n+1}} - L_{\pi}^{\pi^{n+1}} \|$$

$$\leq \gamma \| \pi - \pi^{n+1} \| + \gamma \| \pi^{n+1} - \pi^n \|^2$$

$$= \| L_{\pi}^{\pi} - L_{\pi}^{\pi^{n+1}} \| + \| L_{\pi}^{\pi^{n+1}} - L_{\pi}^{\pi^{n+1}} \|$$

$$\leq \gamma \| \pi - \pi^{n+1} \| + \gamma \| \pi^{n+1} - \pi^n \|^2$$

$$\| \pi - \pi^{n+1} \| \leq \frac{\gamma}{1-\gamma} \| \pi^{n+1} - \pi^n \|^2$$

Simplify second term:

$$\begin{aligned}
 \|v^{n+1} - v^*\| &\leq \sum_{k=0}^{\infty} \|v^{n+k+2} - v^{n+k+1}\| \\
 &= \sum_{k=0}^{\infty} \|L^{k+1} v^{n+1} - L^{k+1} v^n\| \\
 &\leq \sum_{k=0}^{\infty} \gamma^{k+1} \|v^{n+1} - v^n\| \\
 &= \frac{\gamma}{1-\gamma} \|v^{n+1} - v^n\|
 \end{aligned}$$

D - already done is $< \epsilon/2$

Value iteration - using that stopping criterion converges to epsilon optimal policy

- Pick epsilon, you get stopping criteria

ToDo: Read writeup (to be put up)

21min: **Rate of convergence - can study if interested**

- How quickly does it approach

L is contraction mapping

- Contraction factor is gamma
 - For every iterate - successive iterate - gamma closer to v^*
 - Gamma small - converge faster
 - Gamma big - convergences slower
 - This is called Linear Convergence with rate of gamma
- If it falls as gamma square - it is quadratic

w5-L29 Policy Iteration (13min)

start off with some arbitrary policy let us say π_0 may start off with some arbitrary policy π_0 I find V_{π_0} Right then what I do I am greedy with respect to V_{π_0} I will find π_1 right.

for each $s \in \mathcal{S}$,

$$\pi(s) = \arg \max_a \left\{ E[r|s,a] + \gamma \sum_{s'} p(s'|s,a) \cdot V^{n+1}(s') \right\}$$

Keep going until converge

- Rate of convergence is much faster

Value Iteration:

- Favourite for OR people

Policy Iteration

1. Select π_0 ; Set $n=0$
2. (Policy Eval) $V^{\pi_n} = (\mathbf{I} - \gamma P_{\pi_n})^{-1} r_{\pi_n}$

Policy improvement okay this is a second step is called policy evaluation

1. Select π_0 ; Set $n=0$
2. (Policy Eval) $V^{\pi_n} = (\mathbf{I} - \gamma P_{\pi_n})^{-1} r_{\pi_n}$
3. (Policy Impr) Choose π_{n+1}

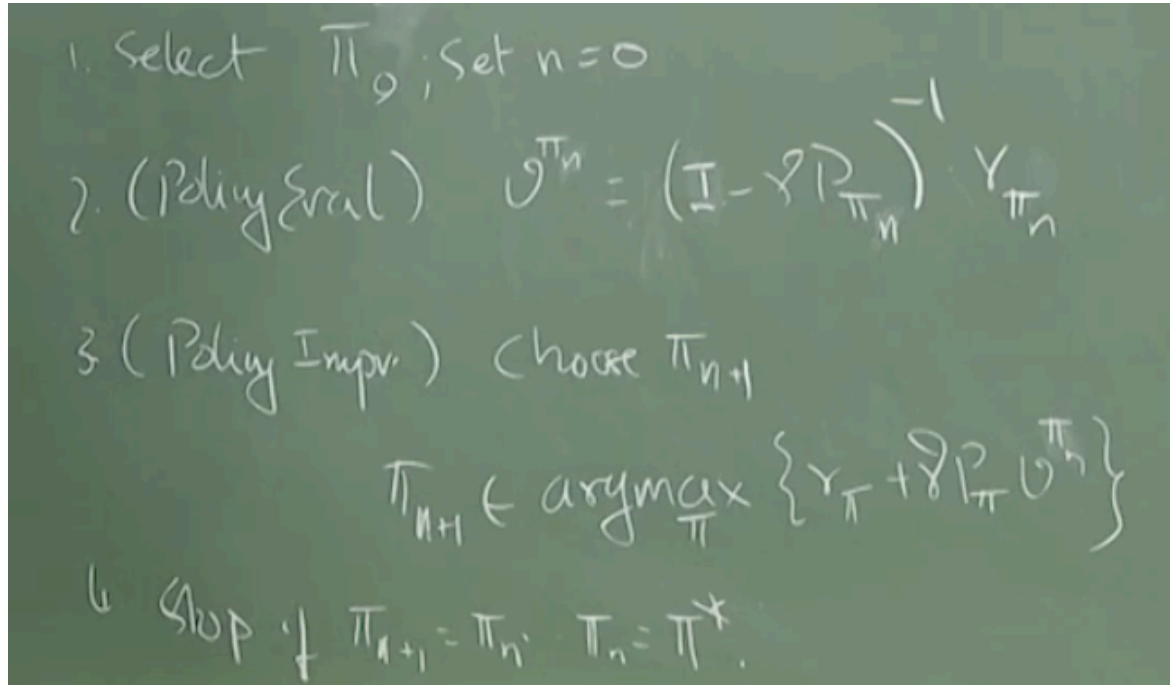
$$\pi_{n+1} \in \arg \max_{\pi} \left\{ r_{\pi} + \gamma P_{\pi} V^{\pi_n} \right\}$$

third step is called policy improvement, argmax is done componentwise

At every step checking for equality - policy for state s

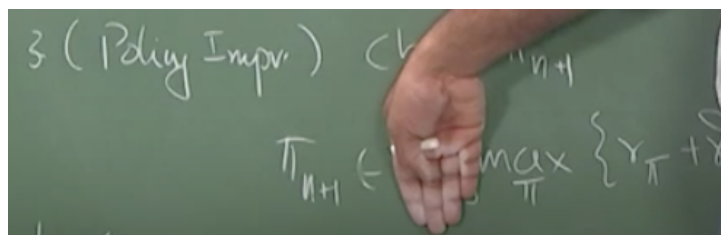
Going up and right both are right

- If in next step chose right - consistently break ties



If stopping condition holds - why π_n should be π^*

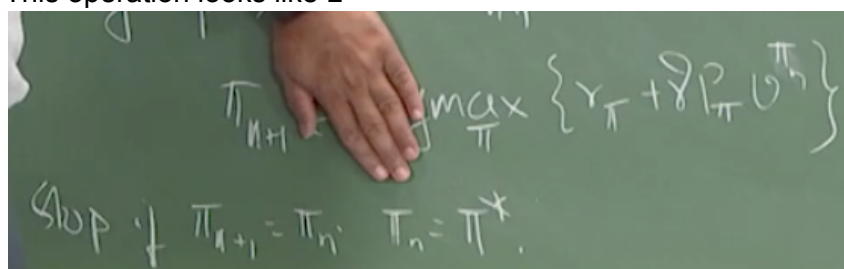
- If i get same π - value fn hasn't change : $\pi_n = \pi_{n+1}$ then π_n is π^*



This part is L operating on

It hasn't changed, so it is optimal value function

This operation looks like L



If i get same π - value function hasn't changed

If $\pi_{n-1} = \pi_{n+1}$ then π_{n-1} is a π^*

I will have to show that happens

Do in two ways:

1. $v_{\pi_{n+1}} \geq v_{\pi_n}$

- why do you think it should be greater

Let π_{n+1} satisfy (3)

π_n is greedy policy

π_{n+1} satisfies 3

Let π_{n+1} satisfy (3)

$$r_{\pi_{n+1}} + \gamma P_{\pi_{n+1}} v^{\pi_n} \geq r_{\pi_n} + \gamma P_{\pi_n} v^{\pi_n} = v^{\pi_n}$$
$$r_{\pi_{n+1}} \geq (I - \gamma P_{\pi_{n+1}}) v^{\pi_n}$$

Left was taken by maximizing - so it should be greater

$$(\mathbf{I} - \gamma P_{\pi_{n+1}})^{-1}$$

Multiply both sides by

Let π_{n+1} satisfy (3) Watch later

$$\gamma_{\pi_{n+1}} + \gamma P_{\pi_{n+1}} v^{\pi_n} \geq \gamma_{\pi_n} + \gamma P_{\pi_n} v^{\pi_n} = v^{\pi_n}$$

$$\gamma_{\pi_{n+1}} \geq (\mathbf{I} - \gamma P_{\pi_{n+1}}) v^{\pi_n}$$

$$(\mathbf{I} - \gamma P_{\pi_{n+1}})^{-1}$$

$$v^{\pi_{n+1}} \geq v^{\pi_n}$$

It is componentwise

You get:

Last thing to show there are finite number deterministic of policies that you can search through

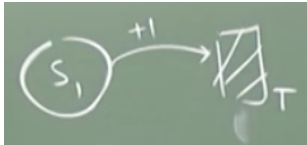
- If you have finite MDP
- So policy iteration will converge

w5-L30 DP (35min)

One step problems - 1st step you get reward, have value for termination

$$V^{n+1}(s) = \max_a \left\{ E[r|s,a] + \gamma \sum_{s'} P(s'|s,a) \cdot V^n(s') \right\}$$

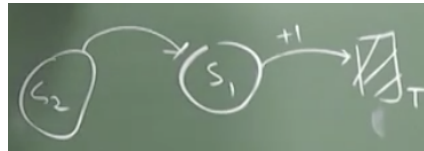
In one step if you reach a terminal state - second term = 0



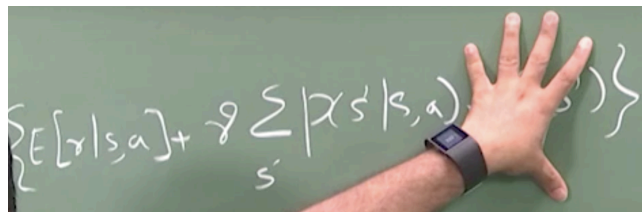
V^{n+1} will be for s_1

in first iteration will compute right value of state

Let us say we have another state:



Two iterations are needed to get value for s_2



This computes step for n-step problem:

DP: subproblem which i solve - nth step prediction

Whole thing computes value for $n+1 \rightarrow$ will be $r + \text{nth step prediction}$ - makes it

Stochastic DP:

$$V^{n+1}(s) = \max_a \left\{ E[r|s,a] + \gamma \sum_{s'} P(s'|s,a) \cdot V^n(s') \right\}$$

I allow you to act 1, 2, 3, upto infinite no. of time steps

That is what value iteration does

Solve for $n+1$

Policy iteration: is DP but little tricky

Asynchronous DP:

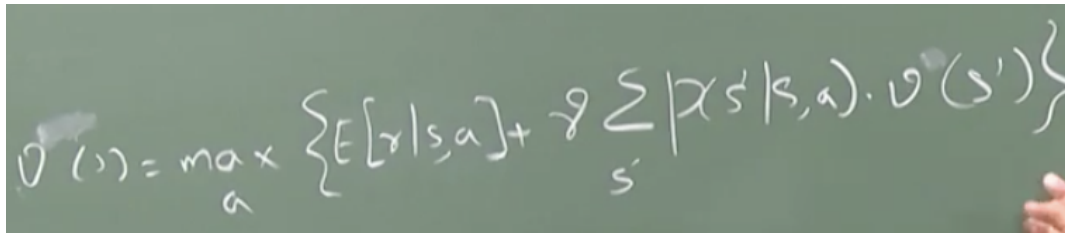
- One step in Value iteration

For all v_{n+1} computation - use v_n on right hand side

- Different array $n+1$, keep doing it

You can do it inplace

- Don't use two arrays - do with one array
- I have to go through some ordering
- **Will use old value of v_{n+1} but new v_n**
 - This is little tricky, no longer clear, what am i doing
- Computing v_{n+1} of 5. I am using further iterate of v_{n+1} , and v_{n+2}
- It will converge
 - I can pick any state and update its value
 - Then randomly pick some other state


$$V^*(s) = \max_a \left\{ E[r|s,a] + \gamma \sum_{s'} P(s'|s,a) \cdot V^*(s') \right\}$$

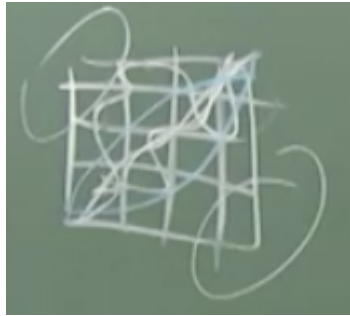
Only condition for convergence:

- Ensure - i sample all the states at least infinitely often
- Powerful result

Real Time DP:

- Almost like RL
- Issues in value/policy iteration - for large state space
 - Every Time you do iteration - you will go through all states
 - That is not often needed
 - Some state are seldom visited - don't mind
 - Often visited states - weigh error with prob of visiting the state $v^*(s)$
 - Why do i want to do it?
 - You don't have infinite computation power
 - If MDP has 100million states
 - Don't even want to run finite no. of time
 - How will i spend
 - Spend more time on states which you will visit often
 - Start from some state - execute some trajectory
 - Look at states which are in trajectory
 - You have made changes to value
 - This implicitly give policy - recover it

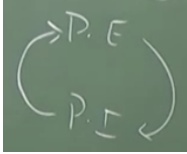
- Now run with this policy - see which states you are visiting
- Run update on those states
- I am not worried about what sample i am doing update
 - Realtime - generating states using current policy
 - As value fn gets closer to optimum, policy also gets closer to optimum
- Set of states those you sample are

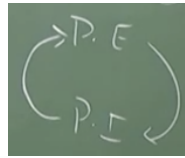


- Highly likely to appear
- Envelope of states - will apply more DP update
- You never behave greedy
 - You behave epsilon greedy
- If model is available
 - It is powerful
- To ensure converge
 - You have to visit each state infinitely often
- Crucial:
 - **We use Sampling to which states to update**
 - **In future RL** - what is the reward with which you have update
 - And what is the terminal cost

RTDP is not in chapter 4: Original paper is much better

Generalised Policy Iteration (GPI):

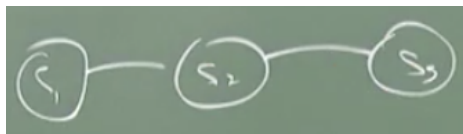
- | | |
|--|--|
| <ul style="list-style-type: none">1. Policy Evaluation2. Policy Improvement <p>Keep doing until policy doesn't change</p> |  |
|--|--|



Ref: MDP - Putterman - don't try to read

Took 3 chapters - cherry picked result - simplified version in class

- Somewhat like Async DP
- I have evaluated policy - don't be greedy on all states
- Do i have to evaluate policy in every state
 - Evaluate on some states
 - and closeby states around the state i change
- With arbitrary π - update trajectories
- Try to be greedy wrt - those states in which you changed value function
 - Will change policy
- Sample more state
 - **Partial policy evaluation and policy improvement**
 - Can save computation
- More useful if you do it on trajectory - why?
 - Pick random set of state - do policy improvement
 - If intersection is empty - don't do imprvment
- RTDP will have greater effect than AsynDP
 - **In trajectory I am sampling state and successors**

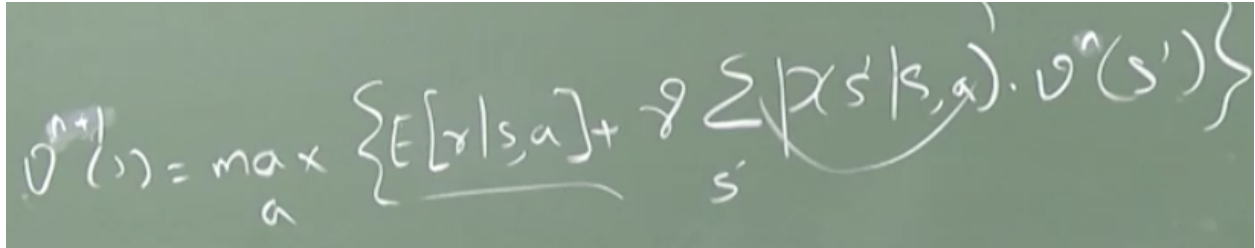


- You don't have to show - samples are along the trajectories
- Rate of convergence improve
 - If you sample along trajectories

Can take it to extreme:

- Instead of taking trajectory - taking on one state at a time
- Take one action according to policy - update value of state
- Be greedy wrt updated value function
- Produce new policy
- Can do with random states or go in schedule

Does it look reasonable?


$$V^{n+1}(s) = \max_a \left\{ E[r|s,a] + \gamma \sum_{s'} P(s'|s,a) \cdot V^n(s') \right\}$$

- Pick best action (max), pick one step (2nd term), what the next thing is (last term)
- This is what value iteration done

Value iteration is extreme form of GPI

Many of the RL algorithms are some form of GPI

Power of GPI:

- As soon as i come up with a mechanism to evaluate policy
- Turn it in a mechanism for learning to solve MDP
- I am going to repeatedly evaluate policies
 - Improve between iterations

This framework will see

1. Evaluation
2. Optimal setup

w5-L31 Monte Carlo (23min)

I don't have access to MDP - Most cases

Cycling: what states and actions are

- Can't think of writing down p (transition dynamics are hard)

$$V^\pi(s) = E_{\pi} \{ G_t \mid s_t = s \}$$

Want to look at $v\text{-}\pi(s)$ -> all over again (assume you didn't know anything about MDP)

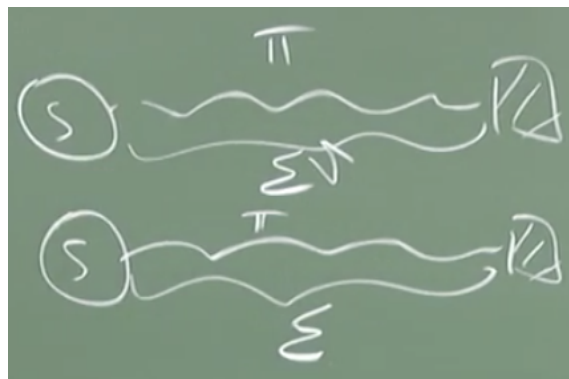
Find Expectation of a random variable:

- Sample lot of times and take average (most straightforward)

Follow policy π until you come to end



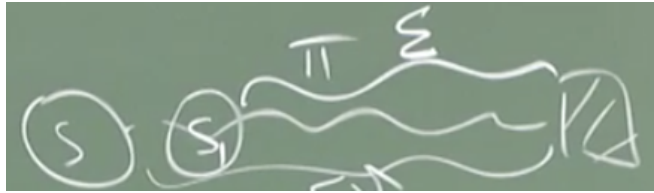
- Sum the rewards - that will give one sample



Start s again - this give another sample
By executing π multiple time

That gives value at state s -> $v\text{-}\pi(s)$

Is that I can do with this trajectory:



Can i use summation for sample s_1

Why is it fine - policy are same, and things are Markov

I generate trajectory

- Look at all states
- Use it to evaluate policies

Monte-Carlo methods (for policy evaluation)

- Using some form of simulations

Note:

$$v^{n+1}(s) = \max_a \left\{ E[r|s,a] + \gamma \sum_{s'} P(s'|s,a) \cdot v^n(s') \right\}$$

- Don't really need the model or reward
- Just should be able to run trajectories

All i need is cycle

- Start paddling
- Don't need to explain p, r,

All you need is infinite no. of cows

You could also get by - real system

Simulation Model / Sample Models:

- Written programs for game
- BlackJack game (as close to 21)
- state space - what cards you see, opponent's cards - you don't see
- Actions: Hit me or Stay
- What card already you have seen
- I can write a program which can play with me
 - Which cards you have dealt Much much more easily than it will compute p

$$Q^*(s,a) = \max_a \left\{ E[r|s,a] + \gamma \sum_{s'} P(s'|s,a) \cdot V^*(s') \right\}$$

Monte-Carlo Policy Evaluation:

Samples I draw

- Trajectories give state and reward
- With which i update value of state

If π is deterministic (fixed - no explore or greedy)

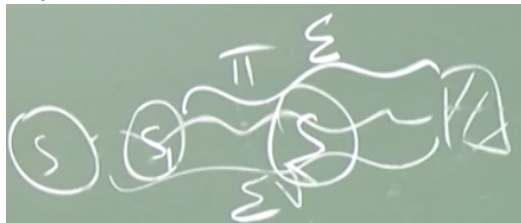
- **What additional thing you have to do**
- Want to find $v_{\pi}(s)$ - **I at least start from each state**

In stochastic policy also you may have to visit each state

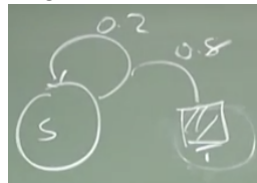
- Unless policy allows you to visit (completely random, with equal prob $1/n$)
 - Entire state space is reachable

Exploring Starts:

- I start from s and run trajectory



- What if s appears again in
- Is it two trajectories starting from s or one trajectory starting from s



- Only one action from s $s \rightarrow s \rightarrow \text{stop}$

How many say 1 trajectory and two trajectories?

- Both are Right

First visit Monte-Carlo: When you do One

Every visit Monte-Carlo: When you do every occurrence

- First return will be full
- Other return will be for sub trajectory

Have on sample trajectory but using part of sample trajectory twice - what is the real problem

- Sample are taken according to the distribution on which i am taking expected
- Distribution yielded sample once - i am using it twice so
- I am changing the distribution - that is main problem
- Not using the same distribution - taking expectation
- All this leads to additional Bias
 - In the limit If you do a lot of samples, both converge to same thing

It affects in some sense:

- Something that happened once in a million times
 - But you will count it twice
 - You hae doubled the probability
- Independent samples will be only once

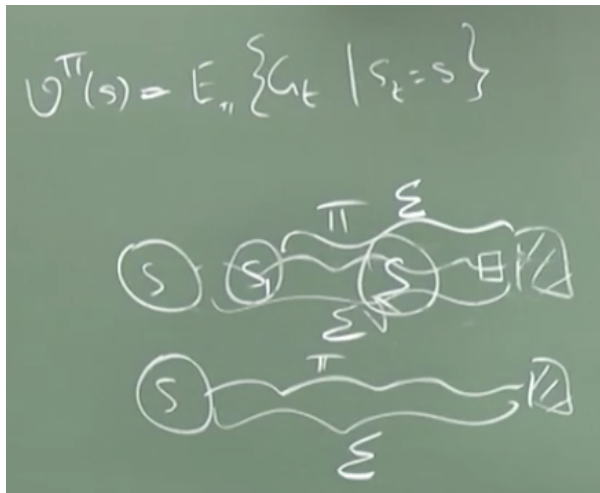
20min: Explanation using dice

w5-L32 Control MC (28min)

1. Evaluation

2. Control

How to improve the policy - how to solve the MDP - learn to control the MDP?



Evaluated on few trajectories - this value is approximation - not true value

You can be greedy wrt - value function?

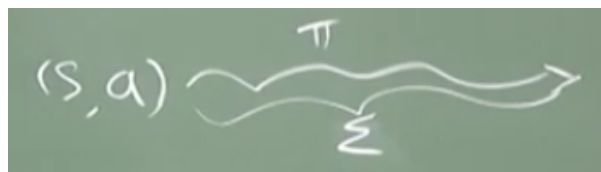
How will you do it?

Don't use v-fn use q-value fn

- Just look at the values of the action

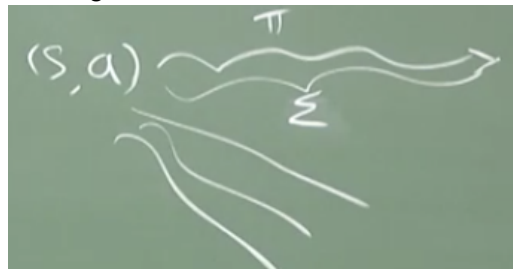
In control - I use q function

How do i do Monte-carlo evaluation for q-fn

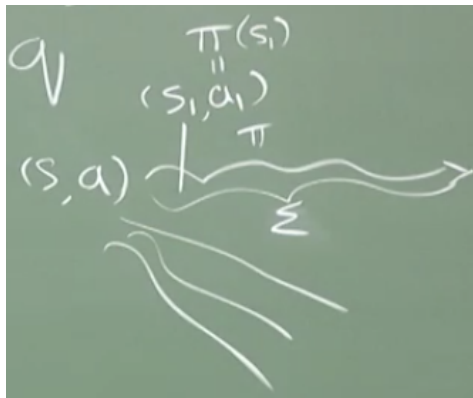


Fix (s,a), follow pi, sum rewards, that gives one sample

Do it again



Fundamental question:



can i consider this as sample take from s_1, a_1

I want to evaluate action pairs (s, a)

If you do this you get in all

- Disproportionate action corresponding to π

All samples are used for (s, a)

- If i need to do s_1, a_1 - follow again from s, a

For long time convergence was open question s_1, a_1

- Proven (in two decades) that it will not converge

Use first state and action

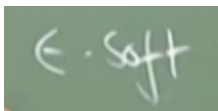
Most case in practice - in practice s_1, a_1 works

Suppose i reach s again (let us not go there)

One way of avoiding exploring starts:

Epsilon-soft policy:

- Mostly greedy
- Mostly prob to one action
- Small prob to other actions
- Not just actions corresponding to π



- Every action has at least prob epsilon of being taken - from every state
- Uniformly random policy is epsilon soft policy - $1/n$ soft policy
- Epsilon greedy execution of policy is also epsilon/n soft policy
- Can apply concentration inequality - this is Markov - π is fixed
 - Here it become tricky keep changing policy

It will converge to epsilon soft policy

- Take greedy, make epsilon greedy
- Keep doing this - converge to something
 - Highest prob will be given to optimal action
 - Proof given in text book

Importance Sampling:

If using - epsilon soft - for non-exploring starts

Deterministic - need to do exploration

I can behave how ever i want to behave - but give you any policy you asked for

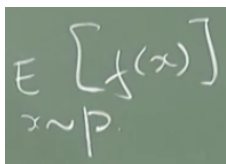
On-policy:

- Samples are generated based on policy being evaluated

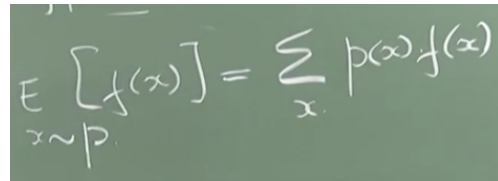
Off-Policy: Decouple

- Policy which is evaluating
- Policy based on which I am behaving

Compute E of some $f(x)$ where x is sample from some distribution p

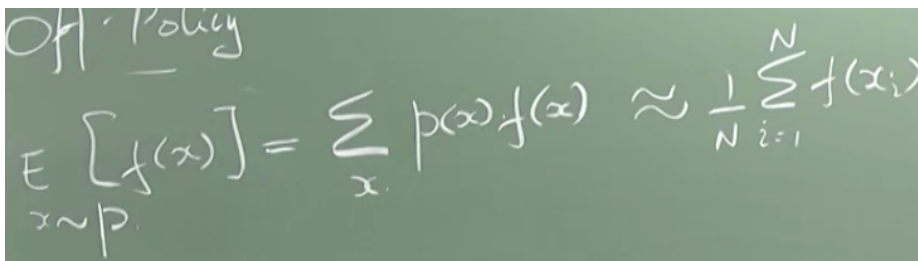

$$E[f(x)]_{x \sim p}$$

it is discrete distribution this is expectation


$$E[f(x)]_{x \sim p} = \sum_x p(x) f(x)$$

Don't have access to p , but can sample - use p , ask to give x

- Plug them in $f(x)$
- Each x_i was sampled from p


$$\text{Off-Policy} \\ E[f(x)]_{x \sim p} = \sum_x p(x) f(x) \approx \frac{1}{N} \sum_{i=1}^N f(x_i)$$

I know something about certain outcome

You can tell me according to q - diff coin you are tossing

$$= \sum_x p(x) \cdot \frac{q(x)}{q(x)} \cdot f(x)$$

3, 4 occur more earlier, then 8, 9 later

$$= \sum_x \frac{p(x)}{q(x)} \cdot f(x) \cdot q(x)$$

Rewrite as:

Off-Policy

$$E_{x \sim p} [f(x)] = \sum_x p(x) f(x) \approx \frac{1}{N} \sum_{i=1}^N f(x_i)$$

$$= \sum_x p(x) \cdot \frac{q(x)}{q(x)} \cdot f(x)$$

$$= \sum_x \frac{p(x)}{q(x)} \cdot f(x) \cdot q(x)$$

$$= E_{x \sim q} \left[\frac{p(x)}{q(x)} \cdot f(x) \right]$$

$f(x)$ is weighed by ratio of probabilities

$$= E_{x \sim q} \left[\frac{p(x)}{q(x)} \cdot f(x) \right] \approx \frac{1}{N} \sum_i f(x_i) \cdot \frac{p(x_i)}{q(x_i)}$$

Average of $f(x)$ if samples are drawn according to p

$$= E_{x \sim q} \left[\underbrace{\left(\frac{p(x)}{q(x)} \right)}_{\text{Importance Factor / Weight}} f(x) \right] \approx \frac{1}{N} \sum_i f(x_i) \cdot \frac{p(x_i)}{q(x_i)}$$

Importance Factor / Weight:

How important is specific x in determining the expectation of $f(x)$ wrt to p , when drawing samples according to q

- Gives relative difference
- If $p(x)$ is large, q is small importance is large
- If $p(x)$ is small, q is large importance is small
- If $p(x)$ and $q(x)$ comparable prob - don't care - use as it is $f(x)$

Why called importance:

x more frequent under q - give lower weight when i compute expectation

x less frequent under q - give more weight when i compute expectation

Determining Expectation = Importance Sampling

Easier to sample from q

- To draw sample if CDF is easier to compute - in such cases do importance sampling

$f(x)$ will become

- Deterministic policy - random behaviour policy
- G_t - return i take from trajectory
- p and q are policies according to which i am behaving

Is there any condition about which we need to care:

- If $p(x)$ is non zero then $q(x)$ has to be non-zero
 - Ensure by epsilon-soft
 - Every action has some small prob of picking

Has large variance

Normalised Importance Sampling:

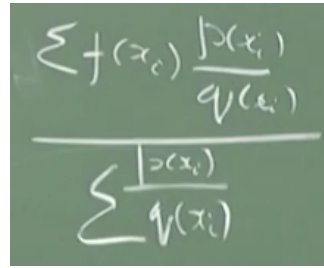
Can suffer from instability

$q(x)$ - close to zero - importance can be very large

- Can add oscillations
- Instead of dividing by N

Ratio will become large in numerator and denominator

Ratio won't oscillate too much


$$\frac{\sum f(x_i) \frac{p(x_i)}{q(x_i)}}{\sum \frac{p(x_i)}{q(x_i)}}$$

Has slightly higher bias

Both converge to same

Normalised Importance Sampling - used in practice

